

```
# Calling with positional arguments
ma_value = calculate_moving_average(apple_prices, 3)
```

Here, `apple_prices` is passed as `prices` and `3` is passed as `window` based on their positions.

Keyword Arguments

Alternatively, you can specify which parameter each argument is for:

```
# Calling with keyword arguments
ma_value = calculate_moving_average(prices=apple_prices, window=3)
```

Keyword arguments make your code more readable, especially when a function has many parameters. They also allow you to provide arguments in any order:

```
# Order doesn't matter with keyword arguments
ma_value = calculate_moving_average(window=3, prices=apple_prices)
```

Default Arguments: Building in Flexibility

Functions can define default values for parameters, which are used when the caller doesn't provide that argument:

```
1. def calculate_moving_average(prices, window=20):
2.     # Default window is 20 if not specified
3.     sum_of_prices = sum(prices[-window:])
4.     moving_average = sum_of_prices / window
5.     return moving_average
6.
7. With this definition, you can call the function with or without specifying a window:
8. # Uses default 20-day window
9. long_term_ma = calculate_moving_average(apple_prices)
10.
11. # Specifies a custom 5-day window
12. short_term_ma = calculate_moving_average(apple_prices, 5)
13.
```

Default arguments are particularly valuable in trading systems, where certain parameters (like lookback periods or thresholds) have standard values but might need adjustment for specific strategies.